

TASK ENGINE API

Moteur d'execution de taches distribue

Celery + FastAPI + SQLite

Broker : SQLite (dev) / Redis (prod)

API : FastAPI - <http://localhost:8000>

Docs : <http://localhost:8000/docs>

Worker : Celery --pool=solo

TABLE DES MATIERES

1. Architecture du projet
2. Démarrage rapide
3. Concepts clés : `handler_url` et `callback_url`
4. Flux d'exécution complet
5. Endpoints API
6. Statuts des tâches
7. Structure des fichiers
8. Ajouter un nouveau projet
9. Exemples de soumission
10. Passer en production

1. Architecture du projet

Le Task Engine est un moteur d'exécution de tâches asynchrones universel. Il reçoit des tâches de n'importe quelle application externe via une API REST, les exécute via des workers Celery, et notifie les applications des résultats.

Structure des composants

```

App Externe (CRM, GEO, Facturation...)
    |
    | POST /tasks/submit { payload, handler_url, callback_url }
    v
+-----+
|   FastAPI (port 8000)   | <- Reçoit et enregistre les tâches
|   api/main.py           |
+-----+
    | send_task("task.execute")
    v
+-----+
| Broker SQLite / Redis   | <- File d'attente des tâches
+-----+
    |
    v
+-----+
| Celery Worker           | <- Exécute les tâches
| tasks/generic.py        |
| tasks/notifier_tasks.py |
+-----+
    |
    +--> POST handler_url (service métier de l'app)
    +--> POST callback_url (notification à l'app)
  
```

Fichiers du projet

Fichier	Role
celery_app.py	Configuration Celery (broker, queues, workers)
api/main.py	Application FastAPI + démarrage DB
api/schemas.py	Modèles de données (Pydantic)
api/routes/tasks.py	Endpoints : submit, status, historique, annuler
api/routes/queues.py	Endpoints : queues, workers stats
core/database.py	Connexion SQLite + session SQLAlchemy
core/models.py	Table tasks (SQLAlchemy ORM)
tasks/generic.py	Tâche universelle task.execute
tasks/notifier_tasks.py	Envoi du callback HTTP
start.ps1	Script de démarrage (worker + API)

2. Demarrage rapide

Prerequis

- Python 3.12+
- Environnement virtuel .venv deja configure (pip install -r requirements.txt)
- Redis optionnel en production (SQLite utilise par default en dev)

Lancer le projet (une seule commande)

```
cd C:\proj_python\celery_proj
.\start.ps1
```

Le script start.ps1 ouvre automatiquement deux fenetres :

- Fenetre Worker : celery -A celery_app worker --loglevel=info --pool=solo
- Fenetre API : uvicorn api.main:app --reload --port 8000

Si start.ps1 est bloque par la politique PowerShell

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Verifier que le worker est actif

```
GET http://localhost:8000/queues/workers/ping
```

Reponse attendue : { "workers": { "celery@PC": { ... } } }

Acces Swagger UI

Tous les endpoints sont testables graphiquement sur :

```
http://localhost:8000/docs
```

3. Concepts clés : handler_url et callback_url

handler_url - Qui traite les données ?

L'URL du service métier de ton application qui reçoit le payload et effectue le vrai traitement (enregistrement BDD, calculs, envoi email, etc.). Le moteur ne connaît pas la logique métier - il délègue simplement.

```
Worker -> POST handler_url
      Body : { "task_id": "...", "task_name": "geo.position", "payload": {...} }
      <- Reponse JSON du service (resultat du traitement)
```

callback_url - Qui est notifié quand c'est fini ?

L'URL appelée automatiquement après l'exécution complète de la tâche. Permet à l'application source de recevoir le résultat sans interroger l'API en boucle.

```
Worker (termine) -> POST callback_url
      Body : {
        "task_id": "fe99bc03-...",
        "status": "SUCCESS",
        "result": { ... resultat ... },
        "tentative": 1
      }
```

Les 4 combinaisons possibles

handler_url	callback_url	Comportement
Non	Non	Stockage brut du payload. Interroger GET /tasks/{id}.
Non	Oui	Stockage + notification de l'app avec le payload.
Oui	Non	Délégation au service métier. Interroger GET /tasks/{id}.
Oui	Oui	Délégation + notification avec le résultat. Mode complet.

Conseil

Pour tester les callbacks sans service réel, utiliser <https://webhook.site> qui génère une URL unique et affiche les requêtes reçues en temps réel.

4. Flux d'execution complet

Scenario : soumission avec handler_url + callback_url

```
1. App externe -> POST /tasks/submit
    {
      "task_name": "geo.position",
      "payload": { "utilisateur_id": "USR-001", "latitude": 5.31 },
      "handler_url": "http://geo-service/process",
      "callback_url": "http://geo-app/webhook",
      "app_source": "geo-tracker-app"
    }

2. API
  -> Genere un task_id UUID
  -> Enregistre en DB (status = PENDING)
  -> Envoie "task.execute" dans le broker Celery
  -> Retourne { "task_id": "abc-123", "status": "PENDING" }

3. Worker
  -> Recoit la tache (status -> STARTED)
  -> POST http://geo-service/process avec le payload
  -> Recoit le resultat du service metier
  -> Sauvegarde le resultat en DB (status -> SUCCESS)
  -> Lance envoyer_callback en arriere-plan

4. Notifier
  -> POST http://geo-app/webhook
    {
      "task_id": "abc-123",
      "status": "SUCCESS",
      "result": { ... resultat du service ... }
    }
  -> callback_status -> SENT
```

Retry automatique en cas d'echec

Tentative	Delai avant retry	Comportement
1	immédiat	Première execution
2	30 sec	Retry 1 - erreur reseau ou timeout
3	60 sec	Retry 2
4	120 sec	Retry 3 - dernier essai
5	-	FAILURE - tache echouee definitivement

5. Endpoints API

Taches

Methode	Endpoint	Description
POST	/tasks/submit	Soumettre une nouvelle tache
GET	/tasks/	Lister l'historique (filtres disponibles)
GET	/tasks/{task_id}	Statut + resultat d'une tache
POST	/tasks/{task_id}/resend	Retenter le callback si echoue
DELETE	/tasks/{task_id}	Annuler une tache

Filtres disponibles sur GET /tasks/

Parametre	Exemple	Description
app_source	geo-tracker-app	Taches d'une application specifique
task_name	geo.position	Taches d'un type specifique
status	FAILURE	Taches par statut
queue	haute_priorite	Taches d'une queue
callback_status	FAILED	Callbacks echoues
limit	50	Nombre de resultats (default: 50)

Queues & Workers

Methode	Endpoint	Description
GET	/queues/	Liste des queues disponibles
GET	/queues/workers	Stats des workers actifs
GET	/queues/workers/ping	Ping pour verifier les workers
GET	/health	Health check

Body de soumission (POST /tasks/submit)

```
{
  "task_name": "geo.position",           // Label libre - identifie le type de tache
  "payload": { ... },                    // Donnees quelconques (structure libre)
  "handler_url": "http://...",           // Optionnel - URL de traitement metier
  "callback_url": "http://...",          // Optionnel - URL de notification
  "app_source": "geo-tracker-app",        // Identifiant de l'application source
  "queue": "default",                    // Optionnel : default | haute_priorite | ...
  "priority": 5,                          // 0 (basse) a 9 (haute) - default: 5
  "countdown": 0,                        // Delai en secondes avant execution
  "expires": null                         // Expiration en secondes (null = jamais)
}
```

6. Statuts des taches

Statuts Celery (champ status)

Statut	Signification	Action possible
PENDING	Soumise, en attente d'un worker	Attendre ou verifier le worker
STARTED	Worker en cours d'execution	Attendre la fin
SUCCESS	Execution reussie	Lire le resultat
FAILURE	Echec apres tous les retries	Consulter l'erreur
RETRY	En attente avant une nouvelle tentative	Attendre le retry
REVOKED	Annulee manuellement	-

Statuts du callback (champ callback_status)

Statut	Signification
null	Pas de callback_url configure
PENDING	Callback en attente d'envoi
SENT	Callback envoye avec succes
RETRYING	Echec temporaire, retry en cours
FAILED	Echec definitif du callback

Recuperer le statut d'une tache

```
GET http://localhost:8000/tasks/{task_id}
```

Reponse :

```
{
  "id": "fe99bc03-3542-4571-a811-5e62d42d2a48",
  "task_name": "geo.position",
  "app_source": "geo-tracker-app",
  "queue": "default",
  "status": "SUCCESS",
  "payload": { "utilisateur_id": "USR-001", ... },
  "handler_url": "http://geo-service/process",
  "result": { ... resultat ... },
  "error": null,
  "priority": 5,
  "callback_url": "http://geo-app/webhook",
  "callback_status": "SENT",
  "created_at": "2026-05-29T08:15:00",
  "updated_at": "2026-05-29T08:15:02"
}
```


8. Ajouter un nouveau projet

Le moteur est universel : aucun code n'est à modifier pour ajouter un nouveau projet. Ton application soumet simplement ses tâches via l'API avec le bon `task_name`.

Etape 1 - Soumettre une tâche

```
POST http://localhost:8000/tasks/submit
{
  "task_name": "mon-projet.mon-action",
  "payload": { ... tes donnees ... },
  "app_source": "mon-application"
}
```

Etape 2 - Exposer un handler dans ton application (optionnel)

```
# Exemple Python / FastAPI cote ton application
@app.post("/api/mon-action/process")
def process(data: dict):
    task_id = data["task_id"]
    payload = data["payload"]

    # Traitement metier
    resultat = faire_quelque_chose(payload)

    return {"status": "ok", "resultat": resultat}
```

Etape 3 - Exposer un webhook pour le callback (optionnel)

```
@app.post("/api/webhook/done")
def on_task_done(data: dict):
    task_id = data["task_id"]
    result = data["result"]
    status = data["status"]

    # Notifier l'utilisateur, mettre à jour l'interface, etc.
    return {"ok": True}
```

Exemple complet - Projet Facturation

```
POST http://localhost:8000/tasks/submit
{
  "task_name": "facturation.facture.generer",
  "payload": {
    "client_id": "CLI-0042",
    "commande_id": "CMD-2026-0187",
    "montant_fcfa": 125000,
    "tva_pct": 18,
  }
}
```


10. Passer en production

Remplacer SQLite par Redis

Redis est recommande en production pour les performances et la fiabilite.

```
# Definir les variables d'environnement avant de lancer
$env:CELERY_BROKER_URL = "redis://:motdepasse@redis-host:6379/0"
$env:CELERY_RESULT_BACKEND = "redis://:motdepasse@redis-host:6379/1"

# Ou avec Docker
docker run -d --name redis -p 6379:6379 redis:alpine
```

Lancer plusieurs workers (Linux/Mac)

```
# Worker avec 4 processus paralleles
celery -A celery_app worker --loglevel=info --concurrency=4

# Workers dedies par queue
celery -A celery_app worker -Q haute_priorite --concurrency=2
celery -A celery_app worker -Q default,basse_priorite --concurrency=4
```

Queues disponibles

Queue	Usage recommande
default	Taches standard - toutes les apps par default
haute_priorite	Taches urgentes - traitement immediat (priority 8-9)
basse_priorite	Traitements longs, batch, non urgents
emails	Envois email - rate limit 100/min
reports	Generation de rapports lourds

Monitorer avec Flower (interface web)

```
pip install flower
celery -A celery_app flower --port=5555
# Interface : http://localhost:5555
```

Checklist production

1. Utiliser Redis comme broker et result backend
2. Configurer au moins 2 workers pour la redondance
3. Activer Flower pour le monitoring
4. Configurer des alertes sur les taches FAILURE
5. Mettre result_expires a 7 jours max pour eviter la saturation